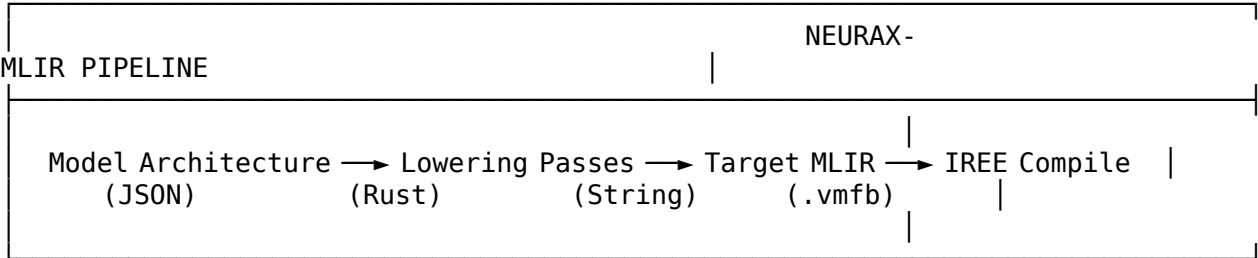


MLIR Code Generation Flow

This document explains how Neurax-MLIR generates code for various hardware targets, from MLIR model definitions to compiled executables.

Overview



The pipeline transforms a high-level model architecture into hardware-specific executable code through several stages:

- 1. **Architecture Definition:** Model structure (layers, dimensions, types)
- 2. **Lowering:** Convert architecture to MLIR operations
- 3. **Target Specialization:** Add hardware-specific attributes and optimizations
- 4. **Compilation:** Generate executable bytecode via IREE

Target Backends

Supported Hardware

Backend	Hardware	Data Types	Key Features
CpuBackend	x86/ARM CPUs	f32, f64, i32, i64	LLVM lowering, vectorization
CudaBackend	NVIDIA GPUs	f16, bf16, f8, f32, i8	Tensor cores, Flash Attention
RocmBackend	AMD GPUs	bf16, f16, f32	MFMA instructions, HIP kernels
MetalBackend	Apple Silicon	f16, f32	M1/M2/M3 GPU optimization
VulkanBackend	Cross-platform GPU	f32	SPIR-V, wide compatibility

Backend Selection

```
use neurax_mlir::{TargetBackend, TargetLowering};

// Select backend based on hardware
let backend = match hardware {
    "nvidia" => TargetBackend::Cuda,
    "amd" => TargetBackend::Rocm,
    "apple" => TargetBackend::Metal,
    "generic-gpu" => TargetBackend::Vulkan,
    _ => TargetBackend::Cpu,
};

// Or auto-detect from GPU name
let backend = TargetBackend::from_str("cuda").unwrap();
```

Code Generation

1. Operator Lowering

Each operator (matmul, conv2d, attention) is lowered to MLIR:

```
use neurax_mlir::{CudaBackend, TargetLowering};

// Generate matmul for CUDA
let matmul = CudaBackend::lower_matmul(
    batch: 1,      // Batch size
    m: 1024,        // Input dimension
    k: 1024,        // Hidden dimension
    n: 1024,        // Output dimension
    dtype: "f16",  // Data type
)?;

// Generate 2D convolution
let conv = CudaBackend::lower_conv2d(
    batch: 1,
    in_channels: 3,
    out_channels: 64,
    height: 224,
    width: 224,
    kernel_size: 7,
    dtype: "f16",
)?;

// Generate multi-head attention
let attn = CudaBackend::lower_attention(
```

```

    seq_len: 2048,
    hidden_size: 8192,
    num_heads: 64,
    dtype: "f16",
)?;

```

2. Generated MLIR Structure

CPU Backend (LLVM)

```

// CPU matmul with LLVM attributes
func.func @matmul(%a: tensor<1x1024x1024xf32>, %b: tensor<1x1024x1024xf32>)
  -> tensor<1x1024x1024xf32> attributes {llvm.readonly} {
  %c_init = tensor.empty() : tensor<1x1024x1024xf32>
  %c = linalg.matmul ins(%a, %b : tensor<1x1024x1024xf32>, tensor<1x1024x1024xf32>)
    outs(%c_init : tensor<1x1024x1024xf32>) -
> tensor<1x1024x1024xf32>
  return %c : tensor<1x1024x1024xf32>
}

```

CUDA Backend (GPU Dialect)

```

// CUDA matmul with GPU kernel attributes
func.func @matmul(%a: tensor<1x1024x1024xf16>, %b: tensor<1x1024x1024xf16>)
  -> tensor<1x1024x1024xf16> attributes {gpu.kernel} {
  %c_init = tensor.empty() : tensor<1x1024x1024xf16>
  %c = linalg.matmul ins(%a, %b : tensor<1x1024x1024xf16>, tensor<1x1024x1024xf16>)
    outs(%c_init : tensor<1x1024x1024xf16>) -
> tensor<1x1024x1024xf16>
  return %c : tensor<1x1024x1024xf16>
}

```

```

// Flash Attention for CUDA
func.func @flash_attention(%q: tensor<2048x64x128xf16>, %k: tensor<2048x64x128xf16>,
  %v: tensor<2048x64x128xf16>)
  -> tensor<2048x64x128xf16> attributes {gpu.kernel} {
  // Fused Q*K^T * V with online softmax
  %output = tensor.empty() : tensor<2048x64x128xf16>
  return %output : tensor<2048x64x128xf16>
}

```

ROCm Backend (AMD GPU)

```

// ROCm matmul with HIP kernel attributes
module @model attributes {
  gpu.container_module,

```

```

    gpu.kernel_attr = "hip"
} {
  func.func @matmul(%a: tensor<1x1024x1024xbf16>, %b: tensor<1x1024x1024xbf16>)
    -> tensor<1x1024x1024xbf16> attributes {gpu.kernel} {
      %c_init = tensor.empty() : tensor<1x1024x1024xbf16>
      %c = linalg.matmul ins(%a, %b) outs(%c_init) -
> tensor<1x1024x1024xbf16>
      return %c : tensor<1x1024x1024xbf16>
    }
}

```

3. Module Generation

Generate a complete MLIR module for a model:

```

// Generate full module for a transformer
let module = generate_cuda_module(
  model_name: "llama-7b",
  hidden_size: 4096,
  num_heads: 32,
  num_layers: 32,
  seq_len: 2048,
  dtype: "f16",
);

```

Output:

```

module @llama-7b attributes {
  gpu.container_module, gpu.kernel_attr = "ptx"
} {
  // Global constants
  %hidden_size = arith.constant 4096 : i64
  %num_heads = arith.constant 32 : i64
  %head_dim = arith.constant 128 : i64
  %seq_len = arith.constant 2048 : i64
  %num_layers = arith.constant 32 : i64

  // RMS Norm (GPU kernel)
  func.func @rms_norm(%input: tensor<2048x4096xf16>, %weight: tensor<4096xf16>)
    -> tensor<2048x4096xf16> attributes {gpu.kernel} {
      %output = tensor.empty() : tensor<2048x4096xf16>
      return %output : tensor<2048x4096xf16>
    }

  // Flash Attention (GPU kernel)
  func.func @flash_attention(...) attributes {gpu.kernel} { ... }

```

```

// MLP with SwiGLU (GPU kernel)
func.func @mlp(...) attributes {gpu.kernel} { ... }

// Forward pass
func.func @forward(%input: tensor<2048x4096xf16>) -
> tensor<2048x4096xf16> {
  %output = tensor.empty() : tensor<2048x4096xf16>
  return %output : tensor<2048x4096xf16>
}

```

Compilation Pipeline

Step 1: Save MLIR

```

use std::fs;

let mlir_code = CudaBackend::lower_matmul(1, 1024, 1024, 1024, "f16")?;
fs::write("model.mlir", &mlir_code)?;

```

Step 2: Compile with IREE

```

# CUDA target
iree-compile model.mlir \
  --iree-hal-target-device=cuda \
  -o model.vmfb

# CPU target
iree-compile model.mlir \
  --iree-hal-target-device=local-task \
  -o model.vmfb

# Vulkan target
iree-compile model.mlir \
  --iree-hal-target-device=vulkan \
  -o model.vmfb

# ROCm target
iree-compile model.mlir \
  --iree-hal-target-device=rocm \
  -o model.vmfb

# Metal target (macOS)
iree-compile model.mlir \
  --iree-hal-target-device=metal \
  -o model.vmfb

```

Step 3: Run Inference

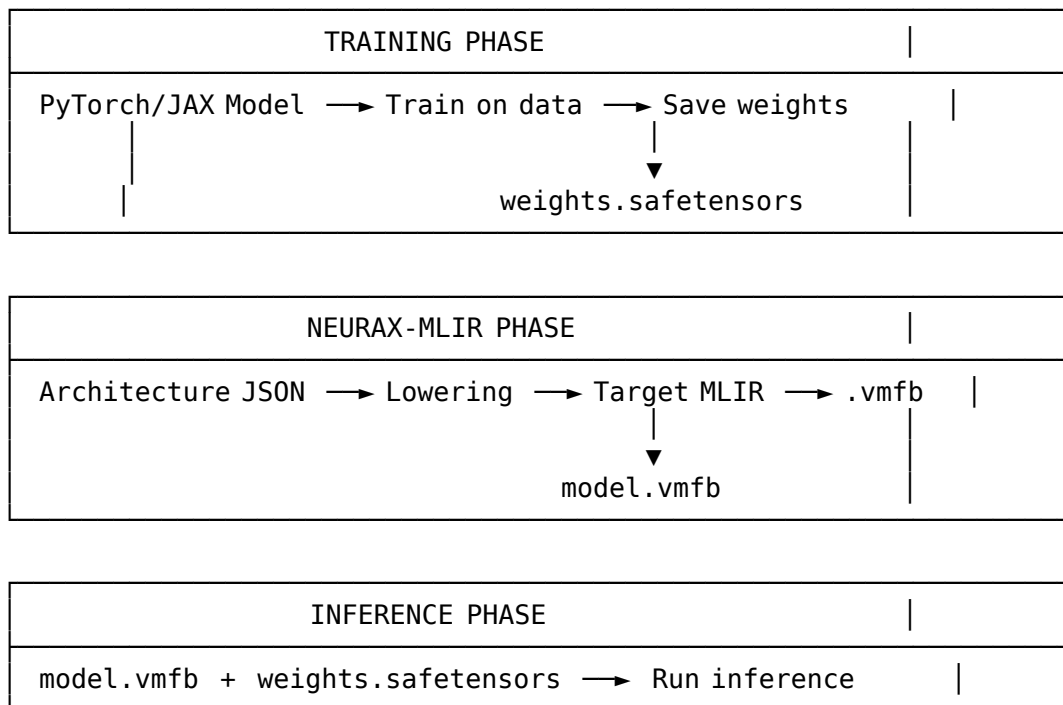
```
# Load weights and run
iree-run-module \
  --device=cuda \
  --module=model.vmfb \
  --parameters=weights.safetensors \
  --function=forward \
  --input="1x2048x164"
```

Integration with Training

The Two-Part Model

Component	Source	Purpose
Architecture (MLIR)	Neurax-MLIR	Defines computation graph
Weights (Safetensors)	Training (PyTorch/JAX)	Contains learned parameters

Training Workflow



Example: Full Pipeline

1. Train model (PyTorch):

```
import torch
from transformers import LlamaForCausalLM

# Train model
model = LlamaForCausalLM.from_pretrained("meta-llama/Llama-7b")
# ... training loop ...

# Save weights
model.save_pretrained("llama-7b-finetuned")
# Creates: pytorch_model.bin or model.safetensors
```

2. Generate matching MLIR (Rust):

```
// Generate architecture that matches the trained model
let mlir = generate_cuda_module(
    "llama-7b-finetuned",
    hidden_size: 4096,
    num_heads: 32,
    num_layers: 32,
    seq_len: 2048,
    dtype: "f16",
);
fs::write("llama.mlir", &mlir)?;
```

3. Compile (Bash):

```
iree-compile llama.mlir --iree-hal-target-device=cuda -o llama.vmbf
```

4. Run inference (Bash):

```
iree-run-module \
  --device=cuda \
  --module=llama.vmbf \
  --parameters=llama-7b-finetuned/model.safetensors \
  --function=forward \
  --input="1x512xi64"
```

Hardware-Specific Optimizations

NVIDIA CUDA

- **Tensor Cores:** WMMA operations for f16/bf16 matmul
- **Flash Attention:** Fused attention kernel with reduced memory
- **Shared Memory:** K/V cache optimization
- **Memory Coalescing:** Layout optimization for bandwidth

```
// Tensor core matmul (16x16x16 blocks)
let tensor_core = generate_tensor_core_matmul(1024, 1024, 1024, "f16");
```

AMD ROCm

- **MFMA Instructions:** Matrix fused multiply-accumulate
- **CDNA Architecture:** Optimized for MI-series GPUs
- **HIP Kernels:** AMD's CUDA-equivalent runtime

```
// MFMA matmul for MI300X
let mfma = generate_mfma_matmul(1024, 1024, 1024, "bf16");

// GPU specs for AMD hardware
let mi300x = AmdGpuSpec::mi300x();
// compute_units: 304, hbm_capacity_gb: 192, memory_bw: 5.3 TB/s
```

Apple Metal

- **Apple Silicon GPUs:** M1/M2/M3 optimization
- **Unified Memory:** CPU/GPU memory sharing
- **SIMD Groups:** Apple's warp equivalent

```
// Apple GPU specifications
let m2_ultra = AppleGpuSpec::m2_ultra();
// gpu_cores: 48, memory_bandwidth: 800 GB/s, tflops_fp16: 15.8
```

Vulkan

- **SPIR-V:** Cross-platform shader format
- **Wide Compatibility:** Works on any Vulkan-capable GPU
- **Portable:** No vendor lock-in

```
// SPIR-V capabilities
let caps = spirv_capabilities();
// Shader, Float16, Int64, GroupNonUniform, ...
```

API Reference

TargetLowering Trait

```
pub trait TargetLowering {
    /// Get the backend type
    fn backend() -> TargetBackend;

    /// Supported data types for this backend
    fn supported_dtypes() -> &'static [&'static str];
}
```



```

/// Lower batched matmul
fn lower_matmul(batch: usize, m: usize, k: usize, n: usize, dtype: &str)
    -> Result<String, String>;

/// Lower 2D convolution
fn lower_conv2d(batch: usize, in_channels: usize, out_channels: usize,
    height: usize, width: usize, kernel_size: usize, dtype: &str)
    -> Result<String, String>;

/// Lower multi-head attention
fn lower_attention(seq_len: usize, hidden_size: usize, num_heads: usize,
    dtype: &str) -> Result<String, String>;

/// Module-level attributes
fn module_attributes() -> String;

/// Function-level attributes
fn function_attributes() -> String;
}

```

TargetBackend Enum

```

pub enum TargetBackend {
    Cpu,      // LLVM CPU lowering
    Cuda,     // NVIDIA GPU
    Vulkan,   // Cross-platform GPU
    Metal,    // Apple Silicon
    Rocm,     // AMD GPU
}

impl TargetBackend {
    /// Check if this is a GPU backend
    pub fn is_gpu(&self) -> bool;

    /// Get IREE target string
    pub fn iree_target(&self) -> &'static str;

    /// Parse from string
    pub fn from_str(s: &str) -> Option<Self>;

    /// List all backends
    pub fn all() -> Vec<Self>;
}

```

Troubleshooting

Common Issues

1. IREE not installed:

Install via pip

```
pip install iree-compiler iree-runtime
```

Or download from GitHub releases

<https://github.com/openxla/iree/releases>

2. CUDA not found:

Ensure CUDA toolkit is installed

```
export CUDA_PATH=/usr/local/cuda
```

```
export PATH=$CUDA_PATH/bin:$PATH
```

3. Metal not available: - Only works on macOS with Apple Silicon -
Requires Xcode Command Line Tools

4. Vulkan driver issues:

Check Vulkan support

```
vulkaninfo
```

Install Vulkan SDK

<https://www.lunarg.com/vulkan-sdk/>

Validation

// Test MLIR syntax with mlir-opt (if installed)

```
fn validate_mlir(mlir: &str) -> Result<(), String> {  
    fs::write("temp.mlir", mlir)?;  
    let output = Command::new("mlir-opt")  
        .args(["temp.mlir", "--verify-diagnostics"])  
        .output()?;  
  
    if output.status.success() {  
        Ok(())  
    } else {  
        Err(String::from_utf8_lossy(&output.stderr).to_string())  
    }  
}
```

References

- MLIR Documentation
- IREE Documentation

- [Linalg Dialect](#)
- [GPU Dialect](#)
- [CUDA Programming Guide](#)
- [ROCm Documentation](#)
- [Metal Performance Shaders](#)